Slides for P3666R1 — Bit-precise integers

Document number:
P3869R0

Date:
2025-10-26

Audience:
SG22

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:
Jan Schultke <janschultke@gmail.com>

Source:
github.com/Eisenwave/cpp-proposals/blob/master/src/bitint-slides.cow

---

> **IMPORTANT:** This document has custom controls:
>
> - →, ↓: go to the next slide
> - ←, ↑: go to previous slide

Bit-precise integers

P3666R1

Introduction

C23 now has `_BitInt` type for N-bit integers (WG14 [N2763], [N2775]):

```
// 8-bit unsigned integer initialized with value 255.
// The literal suffix wb is unnecessary in this case.
unsigned _BitInt(8) x = 0xFFwb;
```

- would be very useful in C++ for > 64-bit computation
- *needs* to be in C++ to call C functions portably!
- efforts to standardize in C++ abandoned (mainly [N1744], [P1889R1] (TS))
- implemented by GCC and Clang; max: `_BitInt(8'388'608)`
- **Now**: figuring out the details

P3666R1 Overview

- **Core design**: copy `_BitInt` from C23 & paste w/ minimal changes:
  - allow `_BitInt(1)` ([N3699] "Integer Sets, v3")
  - change type of `0wb` from `_BitInt(2)` to `_BitInt(1)`
  - allow `enum E : _BitInt(N)` ([N3705], "bit-precise enum")
  - add deduction of `N` from `_BitInt(N)`, other C++ stuff
- **Library design**: copy from C23 & paste unchanged:
  - `<cmath>`: treat *all* integers as `double`
  - `<stdbit.h>`: allow `_BitInt(N)` of same width as standard integer
  - `<stdckdint.h>`: allow `_BitInt(N)` only as target type

P3666R1 Points of contention

- ⚡library class template vs fundamental type
- ⚡undefined behavior on signed integer overflow
- ⚡removing some implicit conversions
- ⚡raising the `BITINT_MAXWIDTH`
- ⚡`_BitInt` keyword and `std::bit_int` alias template
- ⚡(CWG/LWG: use of `_BitInt` keyword in specification)

Library class template vs fundamental type

This was discussed as [P3639R0].

> The WG14 delegation to SG22 believes that the C++ type family that deliberately corresponds to _BitInt (perhaps via compatibility macros) should be... (Fundamental/Library)
>
> | SF | F | N | L | SL |
> |---|---|---|---|---|
> | 8 | 1 | 1 | 0 | 0 |
>
> WG21
>
> | SF | F | N | L | SL |
> |---|---|---|---|---|
> | 4 | 5 | 0 | 0 | 0 |

EWG: consensus to have a fundamental type.

UB on signed integer overflow

- **Problem**: signed integer overflow results in UB
- **Idea**: well-defined for `_BitInt`, e.g. wrapping & erroneous (Rust-style)
- ✓ less UB = more good, could catch bugs
- ✗ inconsistent with C
- ✗ not a general solution (`int` stays unchanged)
- ✗ well-defined (wrapping) behavior not very useful
- ✗ UB is useful for optimization
- **Author position**: problem should be approached more generally

Limiting implicit conversions (1/2)

- **Problem**: lots of bug-prone, questionable implicit conversions
- **Idea**: limit conversions for `_BitInt` (e.g. narrowing)
- ❌ almost impossible to specify without "false positives"
  - many issues due to lack of "untyped literals" like in Rust

```
// Problems with C/C++ interoperable headers:
unsigned _BitInt(8) bit = 0;
unsigned _BitInt(3) max3u = -1;

// Problems with "false rejections" for safe cases:
bit &= 0xf; // int → unsigned _BitInt(8)
```

Limiting implicit conversions (2/2)

```cpp
template<std::integral T>
T div_ceil(T x, T y) { // performs integer division, rounding to +∞
  // ⚠️ Could be mixed-sign comparison:
  bool quotient_positive = (x ^ y) >= 0;
  // ⚠️ Could be mixed-sign comparison
  bool adjust = x % y != 0 && quotient_positive;
  // ⚠️ Could be mixed-sign addition between int (0 or 1)
  //    and unsigned _BitInt(N) "x / y":
  // ⚠️ Could be lossy conversion: int → unsigned _BitInt
  return x / y + int(adjust);
}
```

Author position: at most, pick low-hanging fruits (e.g. `_BitInt` → `bool`)

Raising the `BITINT_MAXWIDTH`

- **Problem**: C only guarantees `_BitInt(64)` / `_BitInt(LLONG_WIDTH)`
  - `_BitInt(128)` and wider are not portable
- **Idea**: mandate e.g. `BITINT_MAXWIDTH >= 65'535` (GCC max.)
- ❌ `BITINT_MAXWIDTH` depends on system ABI:
  - e.g. x86_64 psABI: Clang supports `_BitInt(8'388'608)`
  - e.g. Basic C ABI for WASM: Clang supports only `_BitInt(128)`
- ✔ standardizing high `BITINT_MAXWIDTH` pressures ecosystem
- **Author position**: inherit C restrictions, which are still useful

`_BitInt` keyword and `std::bit_int` alias

- **Problem A**: `_BitInt` useful for C-interoperable headers
  - `signed _BitInt` and `unsigned _BitInt`
  - deducing `N` from `_BitInt(N)`
  - `_Atomic`-like compat. macro for *`bit-int`*`(N)` is convoluted
- **Problem B**: `std::bit_int<N>` is a better fit for C++ than `_BitInt(N)`
- **Author position**:
  - both spellings are useful
  - `_BitInt` should be a keyword

Questions

- Agreement with author position?
  - fundamental type better than class template
  - allow `0wb = _BitInt(1)` and `enum E : _BitInt(N)`
  - keep UB on signed integer overflow
  - keep all implicit conversions (at least in this paper)
  - keep the `BITINT_MAXWIDTH` from C
  - add *both* a `_BitInt` keyword and alias templates (`std::bit_int` and `std::bit_uint`)
- Forward to EWG?

References

[P3639R0] *Jan Schultke.* The _BitInt Debate (2025-02-20)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3639r0.html

[N1744] *Michiel Salters.* Big Integer Library Proposal for C++0x (2005-01-13)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2005/n1744.pdf

[P1889R1] *Alexander Zaitsev et al..* C++ Numerics Work In Progress (2019-12-27)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1889r1%2epdf

[P3639R0] *Jan Schultke.* The _BitInt Debate (2025-02-20)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3639r0.html

[N2763] *Aaron Ballman et al..* Adding a Fundamental Type for N-bit integers (2021-06-21)
https://open-std.org/JTC1/SC22/WG14/www/docs/n2763.pdf

[N2775] *Aaron Ballman, Melanie Blower.* Literal suffixes for bit-precise integers (2021-07-13)
https://open-std.org/JTC1/SC22/WG14/www/docs/n2775.pdf

[N3699] *Robert C. Seacord.* Integer Sets, v3 (2025-09-02)
https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3699.pdf

[N3705] *Phillip Klaus Krause.* bit-precise enum (2025-09-05)
https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3705.htm